

Junior Java Developer Take-Home Assessment

Presented by Iquoc Tran



PARKING LOT MANAGEMENT APPLICATION



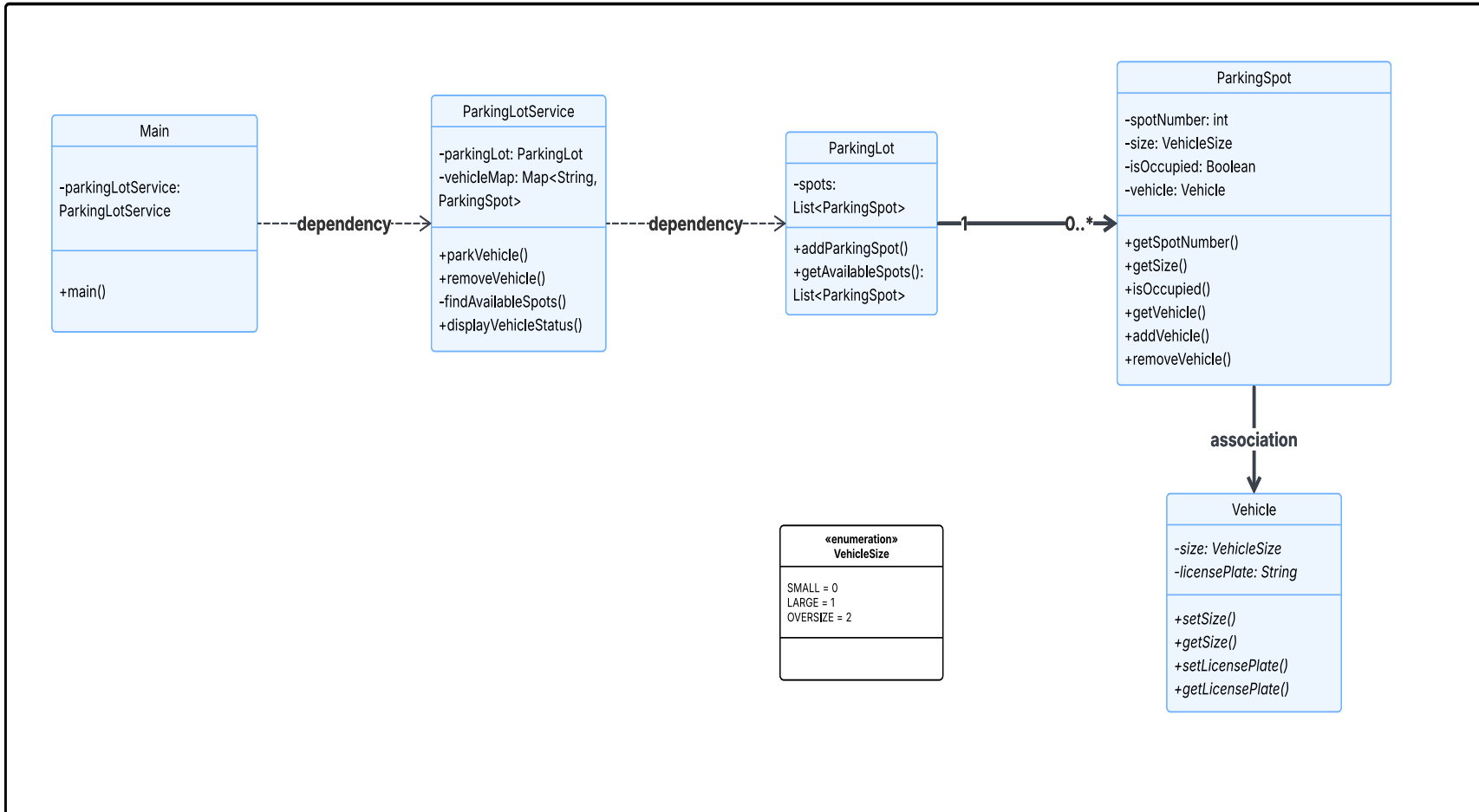
PROJECT OVERVIEW

1. Why Option 1?
 - a. To-Do List Application done in the past
 - b. Inheritance? Encapsulation? Injection? Separation of Concerns?
2. Overall Goal
 - a. Command-line parking lot management system
 - b. Supports parking, removing, and viewing vehicle status
 - c. Handles slot size validation
 - d. Built using object-oriented design principles



Design & Architecture

UML Class Diagram: A-E



- CLI INTERACTION
- HANDLING STATE VERSUS BUSINESS LOGIC
- ENUMERATOR AND RANKING

KEY DESIGN DECISIONS

```
1 package src.models;
2 public enum VehicleSize{
3     SMALL (rank: 1),
4     LARGE(rank: 2),
5     OVERSIZE(rank: 3);
6
7     private final int rank; // rank to determine the size hierarchy
8
9     VehicleSize(int rank) {
10         this.rank = rank;
11     }
12
13     public int getRank() {
14         return rank;
15     }
16 }
```

- Used enums for vehicle and slot sizes
- Used service layer to separate business logic
- Used constructor injection for cleaner architecture and improved testing
- Derived vehicle status via data structures
- Used validation rules for slot compatibility

- ✓ src
 - ✓ models
 - J ParkingLot.java
 - J ParkingSpot.java
 - J Vehicle.java
 - J VehicleSize.java
 - ✓ services
 - J ParkingLotService.java
 - J Main.java
 - ✓ test
 - ✓ models
 - J ParkingLotTest.java
 - J ParkingSpotTest.java
 - J VehicleSizeTest.java
 - J VehicleTest.java
 - ✓ services
 - J ParkingLotServiceTest.java

KEY FILES & FOLDER STRUCTURE

src

Defines behavior

test

Validate behavior

model

Store state

service

Contains business logic

RUNNING THE APPLICATION

Summary of README file found on the Git Repository

Requirements

- JAVA SE Development Kit (JDK) 17 or later installed
- *javac* and *java* available on your system *path*

Build

mkdir out

Compile

javac -d out src\Main.java src\models.java src\services*.java*

Run

java -cp out src.Main

Seed Data

For testing with initialized and preloaded vehicles parked, see lines 23-44 under Main.java

How to Use

- When prompted, enter the number of parking spots to initialize the lot.
- Use the menu options:
 - i. Park a vehicle
 - ii. Remove a vehicle
 - iii. Display vehicle status
 - iv. Exit

Parking input format

- License plate and size, for example:
 - ABC123 SMALL
 - XYZ789 LARGE
 - DEF456 OVERSIZE

*NOT CASE-SENSITIVE

Remove or display input

- Enter only the license plate, for example:
 - ABC123

TESTING & VERIFICATION

Test ID	What was tested	Test Time (05/16/2026)	Result	Bug Severity	Bug Priority	Comment
PL-1	Initialize object with valid input	5:04 PM	PASS			
PL-2	Check if parking spot is added	5:04 PM	PASS			
PL-3	Check if multiple parking spots can be added	5:04 PM	PASS			
PL-4	Check available spots with no added vehicles	5:04 PM	PASS			
PL-5	Check available spots with added vehicles	5:04 PM	PASS			
PL-6	Check empty parking lot	5:04 PM	PASS			The initialization of the parkingSpot array caused errors. FIXED by initializing at global instance instead of constructor.

Test Runner for Java

✓

🔗

ParkingLotTest

\$(symbol-namespace) test.models < \$(project) Parking-Lot-Management-App

✓

📦

testAddMultipleParkingSpots()

\$(symbol-class) ParkingLotTest < \$(symbol-namespace) test.models < \$(project) Parking-Lot-Management-App

✓

📦

testAddParkingSpot()

\$(symbol-class) ParkingLotTest < \$(symbol-namespace) test.models < \$(project) Parking-Lot-Management-App

✓

📦

testEmptyParkingLot()

\$(symbol-class) ParkingLotTest < \$(symbol-namespace) test.models < \$(project) Parking-Lot-Management-App

✓

📦

testGetAvailableSpots()

\$(symbol-class) ParkingLotTest < \$(symbol-namespace) test.models < \$(project) Parking-Lot-Management-App

✓

📦

testGetAvailableSpotsAfterOccupying()

\$(symbol-class) ParkingLotTest < \$(symbol-namespace) test.models < \$(project) Parking-Lot-Management-App

✓

📦

testParkingLotCreation()

\$(symbol-class) ParkingLotTest < \$(symbol-namespace) test.models < \$(project) Parking-Lot-Management-App

> 19 older results

✓

32

@Test

33

public void testAddMultipleParkingSpots() {

34

for (int i = 1; i <= 5; i++) {

35

ParkingSpot spot = new ParkingSpot(i, VehicleSize.LARGE);

36

parkingLot.addParkingSpot(spot);

37

}

38

39

List<ParkingSpot> availableSpots = parkingLot.getAvailableSpots();

40

assertEquals(expected: 5, availableSpots.size());

41

}

SEED DATA

- DISTRIBUTED 50:30:20
- SAMPLE VEHICLE
{ XYZ789 LARGE }

```
Welcome to the Parking Lot Management App!  
Parking lot initialized with 10 slots  
Slot distribution: 5 small, 3 large, 2 oversize
```

```
Sample vehicles added to the parking lot: ABC123 (SMALL), XYZ789 (LARGE), DEF456 (OVERSIZE)  
Vehicle with license plate ABC123 parked in spot 1.  
Vehicle with license plate XYZ789 parked in spot 6.  
Vehicle with license plate DEF456 parked in spot 9.  
Parking lot initialized successfully!
```

```
=== MAIN MENU ===
```

```
Please select an option:  
1. Park a vehicle  
2. Remove a vehicle  
3. Display vehicle status  
4. Exit
```

```
|
```

DEMO 1

INPUT 1 -> 3

INPUT 2 -> XYZ789

```
=== STATUS ===
```

```
Type "back" to return to the main menu.
```

```
Please enter the license plate of the vehicle to display its status:
```

```
XYZ789
```

```
=== VEHICLE PARKING STATUS ===
```

```
License Plate: XYZ789
```

```
Vehicle Size: LARGE
```

```
Parking Spot Size: LARGE
```

```
Parking Spot Number: 6
```

DEMO 2

1. PARKING -> {JEFF2001 LARGE}
2. STATUS -> LARGE SPOT #7
3. REMOVING -> FROM SPOT #7
4. STATUS -> NOT PARKED

```
=== PARKING ===

Type "back" to return to the main menu.
Please enter the license plate and size of the vehicle to park (e.g., ABC123 SMALL)
Sizing options: [SMALL, LARGE, OVERSIZE]
JEFF2001
Invalid input. Type "back" to return to the main menu.
Please enter the license plate and size of the vehicle to park (e.g., ABC123 SMALL)
Sizing options: [SMALL, LARGE, OVERSIZE]
JEFF2001 LARGE
Vehicle with license plate JEFF2001 parked in spot 7.

=== STATUS ===

Type "back" to return to the main menu.
Please enter the license plate of the vehicle to display its status:
JEFF2001
=== VEHICLE PARKING STATUS ===
License Plate: JEFF2001
Vehicle Size: LARGE
Parking Spot Size: LARGE
Parking Spot Number: 7

=== REMOVING ===

Type "back" to return to the main menu.
Please enter the license plate of the vehicle to remove:
JEFF2001
Vehicle with license plate JEFF2001 removed from spot 7.

=== STATUS ===

Type "back" to return to the main menu.
Please enter the license plate of the vehicle to display its status:
JEFF2001
Vehicle with license plate JEFF2001 is not parked in the parking lot.
```

FUTURE IMPROVEMENTS

- REST API with Spring Boot
- Persistent database storage
- GUI/Web Interface
- No best-fit spot assignment
 - SMALL vehicles consume LARGE spots
- Allocating Parking Sizes
 - Random
 - Evenly-distributed
 - Manual-distribution
- Abstraction of Vehicles/Car
 - Sedan: Small
 - SUV: Large
 - Truck: Oversize
- Expand Display Options
 - Taken parking spots versus open parking spots
 - 2D Grid



THANK YOU!

Any questions?